Luchs A protocol reading and decoding examples

Developer guide V0.1 | Source software v1.1.0 | 2026-10-11

Luchs A v1.1 software reference. Wheel commands, signal order and scaling must match the delivered firmware.

Status: Software reference

What a protocol reference tells you

A protocol reference is the dictionary used to interpret CAN messages. The frame ID identifies the message; byte positions identify the signal; type, byte order, scale and offset turn bytes into physical values. It is not a firmware image.

Field	Meaning
Frame ID	Which command or feedback message this is
Signal location	Bytes or bits containing the value
Signedness and byte order	How the raw integer is read
Scale and offset	How a raw value becomes m/s, rad/s, angle or another unit
Direction and period	Who sends the message and how often

For the A v1.1 reference, the source specifies standard 11-bit CAN frames, DLC 8, 1 Mbit/s and big-endian multi-byte values. A DBC is a machine-readable form of this dictionary.

Find the main messages

ID	Direction	What it carries
0x5A0	Host to chassis	Requested body velocity, yaw rate, motion model and control mode
0x5A1	Host to chassis	Gear and axle-angle request
0x5A2 / 0x5A3	Host to chassis	4WS wheel commands or lamps; assignment depends on firmware
0x5B5 / 0x5B6	Chassis to host	Motion and gear/axle feedback
0x5B7 / 0x5B9	Chassis to host	Wheel angles and speeds; verify ordering/scales
0x5B8	Chassis to host	Protection state, a fault-code byte and ultrasonic readings
0x5BA / 0x5BB / 0x5BC	Chassis to host	Fault words, battery and lamp feedback

Decode a value without sending a command

The source defines the forward-velocity field of 0x5A0 as a signed 16-bit value in bytes 0-1, with factor 0.001 m/s and zero offset. This arithmetic example is for reading a capture:

Captured bytes	Raw signed value	Physical value
00 64	100	100 x 0.001 = 0.100 m/s
FF 9C	-100	-100 x 0.001 = -0.100 m/s

Do not apply that factor to wheel speed, battery or an unrelated frame. Each signal needs its own definition. Use a decoder loaded with the matching DBC and compare raw values with the driver output.

```
luchs_cli monitor -i can0 --raw
```

The CLI is part of the source C++ SDK and must be built before use. This monitor command is passive; it is not a drive command.

Keep firmware-dependent settings explicit

Parameter group	Why confirmation matters
wheel_steer_cmd_id	The 0x5A2 / 0x5A3 wheel/lamp assignment can differ
wheel_steer_cmd_order / wheel_steer_fb_order	A correct angle in the wrong slot reaches the wrong wheel
wheel_speed_fb_order / wheel_speed_scale	Ordering and units affect odometry
ultrasonic_scale / battery_*_scale	An assumed scale can produce plausible but incorrect readings

Store confirmed settings with the unit and firmware revision. Use the source protocol sheet and DBC as references; commissioning is a controlled hardware task, not a requirement to follow an unverified motion example on this page.

Scope note. This guide adds explanations and passive decoding examples. It does not alter the existing A protocol files, resolve firmware questions or provide service calibration.

Source and edition

Prepared and reorganised by RJ Tech from the already-public Luchs ROS 2 v1.1.0 package by TerraAgent Robotics. Source licence: Apache-2.0. Source text and examples were shortened, explained and scoped for this topic. Original archives and protocol files remain unchanged.

Source file: docs/protocol_luchs_a.md

Source file: dbc/luchs_a_can_v1.1.dbc

Source file: luchs_driver/README.md

Original public software archive | Source licence